Base de datos 2

Iris Ayala – 23965

Jonathan Díaz- 23837

Anggie Quezada – 23643

Proyecto #1

Sistema de Gestión de Pedidos y Reseñas de Restaurantes

Diseño del Esquema Documental

Descripción del Problema, Alcance y Supuestos

Se requiere diseñar e implementar un Sistema de Gestión de Pedidos y Reseñas de Restaurantes. El sistema debe permitir a usuarios finales registrar restaurantes, realizar pedidos de platillos del menú y dejar reseñas tanto de restaurantes como de pedidos específicos, todo gestionado mediante un backend funcional que utilice MongoDB como base de datos principal.

Los actores principales del sistema son:

- **Cliente:** Buscar, ordenar, reseñar
  - Lee restaurantes, menu_items; escribe ordenes, resenas
- **Propietario:** Gestionar restaurante, menú, ver pedidos
  - Escribe restaurantes, menu_items; lee ordenes de su restaurante
- **Admin:** Moderar, ver reportes globales
  - Lee/escribe todo; ejecuta aggregations complejas
- **Sistema:** Cambiar estados, actualizar contadores,
  - updateOne, bulkWrite, transacciones atómicas

Alcance

- Registro y gestión de usuarios con roles: cliente, propietario y admin.
- Gestión completa de restaurantes: datos, ubicación geoespacial, horario e imágenes.
- Gestión del menú por restaurante: artículos, categorías, etiquetas y disponibilidad.
- Ciclo de vida completo de órdenes: creación, cambios de estado y cancelación.

- Sistema de reseñas vinculables a restaurante y/o a una orden específica.
- Almacenamiento de imágenes (portada, platillos) y exportaciones via GridFS.
- Reportes analíticos: restaurantes mejor calificados, platillos más vendidos, ingresos por mes.

Supuestos de Diseño

- Una orden pertenece a exactamente un restaurante y un usuario.
- Los precios de los items en una orden son snapshots inmutables al momento de la compra.
- El historial de estados de una orden tiene máximo 5 transiciones (ciclo de vida acotado).
- Un usuario puede guardar hasta 10 direcciones (array embebido, tamaño acotado).
- calificacion_prom y total_resenas en restaurantes son campos desnormalizados, actualizados en cada reseña.
- veces_ordenado en menu_items es un contador desnormalizado incrementado con bulkWrite al crear cada orden.
- Una reseña puede apuntar a restaurante, a una orden, o a ambos, al menos uno debe estar presente.
- El carrito de compras es estado de sesión del cliente (frontend), no persiste en MongoDB.
- El sistema rechaza cualquier query sin índice en desarrollo (notablescan activado).

Modelo de Datos Detallado

Colección: usuarios

Campo	Tipo BSON	Índice	Descripción
_id	objectId	PK	Generado por MongoDB
nombre	string	Text	Nombre completo. minLength:2, maxLength:100
email	string	Único simple	Único. Usado para login.
password	string	—	Bcrypt hash. Nunca se proyecta al cliente.
telefono	string	—	Formato E.164. Opcional.
rol	string	Simple	enum: cliente | propietario | admin
activo	bool	—	Soft delete. Default: true.
fecha_registro	date	—	ISODate. Default: Date.now.
preferencias	array<str>	Multikey	Tags: ['vegano','sin_gluten']. Filtra sugerencias.

direcciones	array<obj>	Multikey(ciudad)	DireccionUsuario embebido.

Sub-schema: DireccionUsuario (embebido en usuarios.direcciones[])

Campo	Tipo	Descripción
alias	string	'Casa', 'Trabajo'. Etiqueta para la UI.
calle	string	Requerido.
ciudad	string	Requerido. Indexado vía multikey.
pais	string	Default: GT.
es_principal	bool	Solo una dirección puede ser true por usuario.
coords	GeoJSON	{type:'Point', coordinates:[lng,lat]}. Opcional.

2.2 Colección: restaurantes

Campo	Tipo BSON	Índice	Descripción / Constraint
_id	objectId	PK	Generado por MongoDB
propietario_id	objectId	Compuesto	FK → usuarios._id
nombre	string	Text	minLength:2, maxLength:120
descripcion	string	Text	Descripción visible al cliente.
ubicacion	GeoJSON	2dsphere	{type:'Point', coordinates:[lng,lat]}
direccion	object	—	Embebido: {calle, ciudad, pais, codigo_postal}
categorias	array<str>	Multikey	['italiana','pizza']. minItems:1
horario	object	—	Embebido: {lunes:{abre,cierra,cerrado},...}
telefono	string	—	Contacto directo.
img_portada_id	objectId	—	Ref GridFS fs.files._id
calificacion_prom	double	Simple desc	DESNORM. Default:0. min:0, max:5
total_resenas	int32	—	DESNORM. Default:0. $inc con cada reseña.
activo	bool	Compuesto	Soft delete. Default: true.
fecha_creacion	date	—	ISODate. Default: Date.now.

Sub-schema: Horario (embebido en restaurantes.horario)

Campo	Tipo	Descripción
lunes ... domingo	Object	{abre:'08:00', cierra:'22:00', cerrado:false}. Uno por día.

Colección: menu_items

Campo	Tipo BSON	Índice	Descripción / Constraint
_id	objectId	PK	Generado por MongoDB

restaurante_id	objectId	Compuesto	FK → restaurantes._id. ESR aplicado.
nombre	string	Text	minLength:2
descripcion	string	Text	Ingredientes, alérgenos.
precio	decimal128	—	Decimal128 para precisión monetaria. min:0
categoria	string	Compuesto	enum: entrada|principal|postre|bebida|extra
etiquetas	array<str>	Multikey	['vegano','sin_gluten','picante']
imagen_id	objectId	—	Ref GridFS fs.files._id
disponible	bool	Compuesto	Default: true.
veces_ordenado	int32	—	DESNORM. Default:0. $inc en bulkWrite.
orden_display	int32	—	Posición en el menú. Sort ascendente.
fecha_creacion	date	—	ISODate. Default: Date.now.

Colección: ordenes

Campo	Tipo BSON	Índice	Descripción / Constraint
_id	objectId	PK	Generado por MongoDB
usuario_id	objectId	Compuesto	FK → usuarios._id
restaurante_id	objectId	Compuesto	FK → restaurantes._id
items	array<obj>	Multikey	SNAPSHOT ItemOrden embebido. minItems:1
estado	string	Simple	enum: pendiente|en_proceso|en_camino|entregado|cancelado
historial_estados	array<obj>	—	EstadoLog embebido. Max 5. $push al cambiar.
total	decimal128	—	Suma de subtotales. Calculado al crear.
direccion_entrega	object	—	SNAPSHOT de dirección al crear la orden.
notas	string	—	Instrucciones especiales. maxLength:500
fecha_creacion	date	Compuesto	ISODate. Default: Date.now.
fecha_entrega_real	date	—	Null hasta estado=entregado.
tiene_resena	bool	—	DESNORM. Default:false. Evita $lookup.

Sub-schema: ItemOrden (embebido en ordenes.items[])

Campo	Tipo	Descripción
item_id	objectId	Ref → menu_items._id. Para aggregations.
nombre	string	SNAPSHOT: nombre al momento del pedido.
precio_unitario	decimal128	SNAPSHOT: precio al momento del pedido. Inmutable.
cantidad	int32	min:1
subtotal	decimal128	precio_unitario × cantidad. Calculado.
notas_item	string	'sin cebolla', 'extra queso'. Opcional.

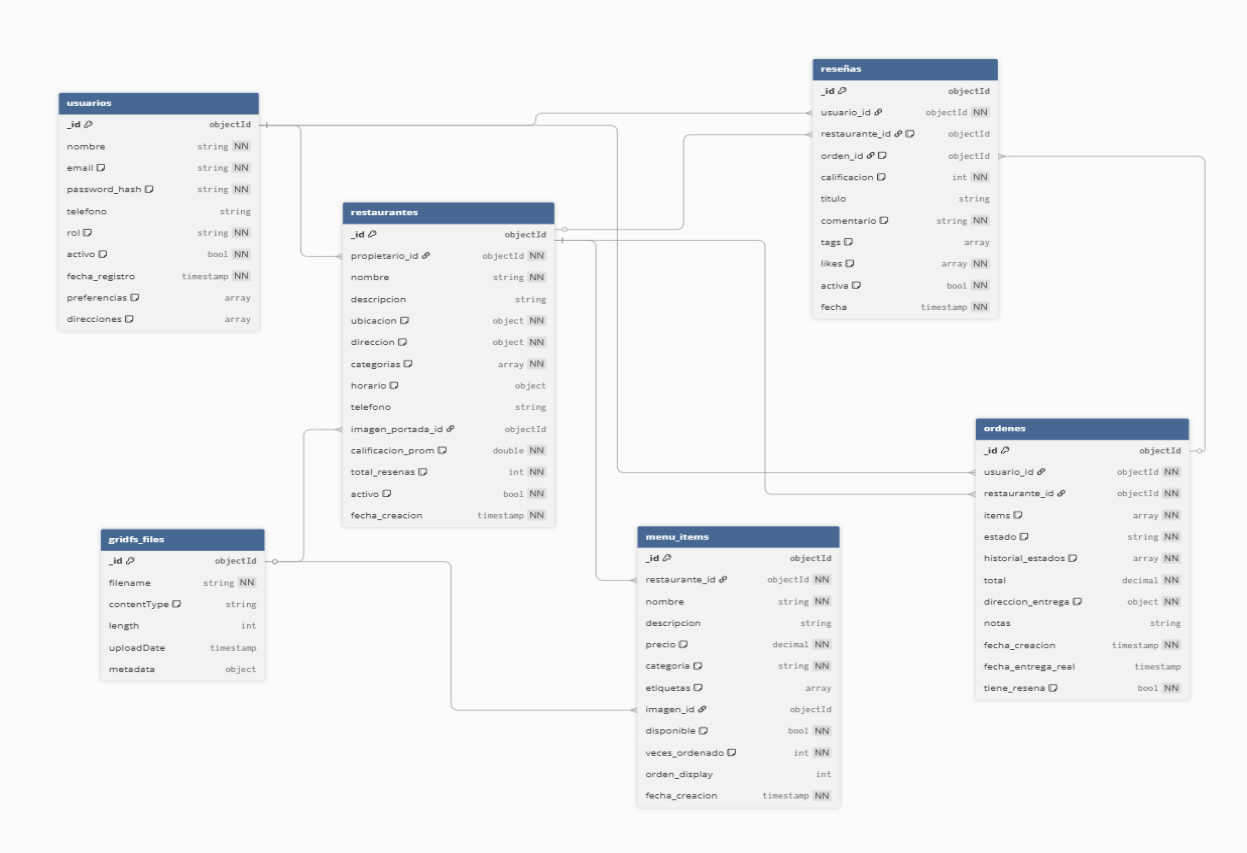
Sub-schema: EstadoLog (embebido en ordenes.historial_estados[])

Campo	Tipo	Descripción
estado	string	Valor del estado en este momento.
timestamp	date	Cuándo ocurrió el cambio.
actor_id	objectId	Quién realizó el cambio (usuario o sistema).
nota	string	Ej: 'Repartidor asignado: Carlos'. Opcional.

Colección: reseñas

Campo	Tipo BSON	Índice	Descripción / Constraint
_id	objectId	PK	Generado por MongoDB
usuario_id	objectId	Simple	FK → usuarios._id
restaurante_id	objectId	Compuesto	FK opcional → restaurantes._id
orden_id	objectId	Simple	FK opcional → ordenes._id
calificacion	int32	Compuesto	1–5 estrellas. min:1, max:5
titulo	string	Text	Max 100 chars.
comentario	string	Text	Min 10 chars.
tags	array<str>	Multikey	['rapido','sabroso','limpio']
likes	array<oid>	Multikey	ObjectIds de usuarios. $addToSet/$pull.
activa	bool	—	Soft delete. Default: true.
fecha	date	Simple desc	ISODate. Default: Date.now.

Diagrama de Relaciones entre Colecciones



Justificación: Embedding vs Referencing

Dato	Decisión	Patrón	Justificación
ordenes.items[]	EMBED	Snapshot	Precio/nombre deben ser inmutables al momento de la compra. Si el propietario cambia el precio, las órdenes pasadas no cambian.
ordenes.historial_estados[]	EMBED	Array acotado	Max 5 transiciones. Siempre se lee junto a la orden. No tiene uso independiente.
ordenes.direccion_entrega	EMBED	Snapshot	La dirección de entrega es un hecho histórico. Si el usuario cambia su dirección después, la orden no debe verse afectada.
restaurantes.horario	EMBED	Objeto fijo	Siempre se lee junto al restaurante. Tamaño acotado (7 días). No tiene uso independiente.
restaurantes.direccion	EMBED	Objeto fijo	Atributo del restaurante. Siempre se muestra junto a él. No se reutiliza.
usuarios.direcciones[]	EMBED	Array acotado	Max 10 elementos. Leídas en cada sesión. Multikey index sobre ciudad.
usuarios.preferencias[]	EMBED	Array strings	Tags simples de tamaño acotado. Leídos en cada petición de personalización.

resenas.likes[]	EMBED	Array ObjectIds	$addToSet (sin duplicados) y $pull son nativos y atómicos sobre arrays. Contar con $size. Razonablemente acotado.
restaurantes ← menu_items	REF	1 col. N	Menú puede tener cientos de items. Paginación propia. Updates individuales. Índice Text independiente. Riesgo de superar 16MB si fuera embebido.
ordenes.usuario_id	REF	ObjectId FK	El usuario tiene ciclo de vida independiente. Su información puede cambiar. Solo se necesita el ID para $lookup cuando sea necesario.
ordenes.restaurante_id	REF	ObjectId FK	Mismo razonamiento que usuario_id. El restaurante puede cambiar nombre, calificación. La orden solo necesita la referencia.
resenas → restaurante_id	REF	ObjectId FK	Una reseña puede no tener restaurante_id. Ciclos de vida independientes.
restaurantes.calificacion_prom	DENORM	Campo calc.	Se muestra en cada card del listado. Recalcular con $avg en cada request sería inaceptable. Se actualiza solo al crear una reseña.
menu_items.veces_ordenado	DENORM	Contador	Dashboard y reportes lo consultan constantemente. bulkWrite lo incrementa en cada orden. Evita $unwind costoso de millones de órdenes.
ordenes.tiene_resena	DENORM	Flag bool	Evita $lookup a resenas para saber si una orden ya fue reseñada al listar el historial del cliente.

Descripción de transacciones implementadas

Transacción: Crear Orden

Colecciones: ordenes (insert) + menu_items (bulkWrite). Requiere sesión MongoDB con writeConcern: majority y readConcern: snapshot.

1. Verificar disponible:true para cada item_id: Si un item ya no está disponible al momento de confirmar, la orden no debe crearse.
2. Recalcular total con precios actuales de BD: El precio en el carrito del cliente puede estar desactualizado respecto a la base de datos.
3. insertOne(ordenes) con snapshot de items y total: El documento debe existir antes de que el cliente vea la confirmación.

4. bulkWrite: $inc veces_ordenado en cada menu_item: Los contadores desnormalizados deben reflejar inmediatamente la nueva orden.
5. COMMIT si todo OK — ROLLBACK completo si falla algún paso: Sin esto, podrían quedar órdenes sin contadores o contadores incrementados sin orden.

Transacción: Cancelar Restaurante

Colecciones: restaurantes + menu_items + ordenes. Afecta 3 colecciones, requiere sesión.

1. restaurantes.updateOne: activo → false: Afecta 1 documento.
2. menu_items.updateMany: disponible → false: Afecta N items del restaurante.
3. ordenes.updateMany: estado → cancelado (solo pendiente/en_proceso): Afecta M órdenes activas del restaurante.
4. $push historial_estados con nota de cancelación: Afecta las mismas M órdenes. Permite trazabilidad del motivo.
5. COMMIT si todo OK — ROLLBACK completo si falla: Garantiza integridad en cascada entre las 3 colecciones.

Explicación detallada de aggregation pipelines

Top Restaurantes por Calificación

Parte de reseñas y no de restaurantes porque necesita recalcular el promedio desde la fuente de verdad, no del campo desnormalizado.

- $match — Filtra reseñas activas con restaurante_id presente. Va primero para reducir volumen antes de agrupar. Usa el índice { restaurante_id, calificacion }.
- $group — Agrupa por restaurante_id, calcula promedio con $avg y cuenta reseñas con $sum: 1.
- $match (segundo) — Filtra grupos con al menos 5 reseñas. No puede ir antes porque necesita el conteo calculado en el $group.
- $sort / $limit — Ordena por promedio desc, desempata por total de reseñas, corta en 10.
- $lookup — Trae nombre, categorías y ubicación del restaurante. Usa sub-pipeline para proyectar solo lo necesario y no arrastrar todo el documento.
- $unwind + $project — Convierte el array del lookup a objeto y redondea el promedio a 2 decimales con $round.

Platillos Más Vendidos

El campo veces_ordenado en menu_items existe para evitar correr este pipeline en cada request del dashboard — un simple find().sort({ veces_ordenado: -1 }).limit(5) es suficiente para el caso común. Este pipeline es la alternativa analítica que además calcula ingresos reales, algo que el campo desnormalizado no puede hacer.

- $match — Solo órdenes entregado y opcionalmente filtrado por restaurante_id. Crítico para reducir volumen antes del $unwind.
- $unwind — Descompone el array items. Una orden con 3 platillos se convierte en 3 documentos. Es la etapa más costosa, por eso el $match anterior importa.
- $group — Agrupa por item_id, suma cantidades con $sum: '$items.cantidad' y calcula ingresos con $multiply precio × cantidad. Toma el nombre del snapshot con $first — no necesita $lookup porque el nombre ya está guardado en la orden.
- $sort / $limit / $project — Ordena por vendidos desc, corta en 10, redondea ingresos.

Ingresos por Restaurante por Mes

- $match — Solo órdenes entregado.
- $group — El _id es un objeto compuesto con restaurante_id, $year y $month de fecha_creacion. Crea un grupo único por restaurante + año + mes. Convierte Decimal128 con $toDecimal antes de sumar porque MongoDB no acumula ese tipo directamente con $sum.
- $sort — Año desc, mes desc, ingresos desc. El reporte más reciente aparece primero.
- $lookup + $project — Trae el nombre del restaurante y construye el campo periodo concatenando año y mes con $concat y $toString, por ejemplo "2026-3".

Este pipeline no se desnormaliza porque los ingresos por mes son una dimensión temporal que crece indefinidamente, no tiene sentido mantener un campo por cada combinación restaurante + mes.

Usuarios con Mayor Gasto Acumulado

Parte de ordenes y no de usuarios porque los datos de gasto viven ahí. Partir al revés obligaría a un $lookup inverso mucho más costoso.

- $match — Solo órdenes entregado.

- $group — Agrupa por usuario_id, cuenta pedidos con $sum: 1 y acumula gasto convirtiendo Decimal128 con $toDecimal antes de sumar.
- $sort / $limit — Top 10 por gasto desc.
- $lookup — Trae nombre y email del usuario con sub-pipeline que excluye explícitamente password_hash, direcciones y demás campos sensibles.
- $project — Usa $arrayElemAt con índice 0 para extraer el objeto del array que devuelve el lookup, en lugar de quedarse con un array de un solo elemento.

Índices definidos y análisis de rendimiento con explain()

Colección	Campos	Tipo	Query que lo justifica
usuarios	{ email: 1 }	Simple/Único	Login: find({ email:'x@x.com' })
usuarios	{ 'direcciones.ciudad': 1 }	Multikey	find({ 'direcciones.ciudad': 'Guatemala' })
usuarios	{ nombre: 'text' }	Text	$text:{ $search:'carlos' }
restaurantes	{ ubicacion: '2dsphere' }	Geoespacial	$near:{ $geometry:Point, $maxDistance:2000 }
restaurantes	{ categorias: 1 }	Multikey	find({ categorias:{ $in:['italiana'] } })
restaurantes	{ propietario_id:1, activo:1 }	Compuesto	Dashboard propietario: sus restaurantes activos
restaurantes	{ calificacion_prom: -1 }	Simple	sort({ calificacion_prom:-1 }) en listado
restaurantes	{ nombre:'text', descripcion:'text' }	Text	$text:{ $search:'pizza guatemalteca' }
menu_items	{ restaurante_id:1, categoria:1, disponible:1 }	Compuesto	find({ restaurante_id, categoria, disponible:true })
menu_items	{ etiquetas: 1 }	Multikey	find({ etiquetas:'vegano' })
menu_items	{ nombre:'text', descripcion:'text' }	Text	$text:{ $search:'pollo sin gluten' }
ordenes	{ usuario_id:1, estado:1, fecha_creacion:-1 }	Compuesto	Historial cliente por estado — ESR rule
ordenes	{ restaurante_id:1, estado:1, fecha_creacion:-1 }	Compuesto	Dashboard propietario — ESR rule
ordenes	{ estado: 1 }	Simple	Filtro admin global por estado

ordenes	{ 'items.item_id': 1 }	Multikey	Aggregation platillos vendidos — $unwind items
reseñas	{ restaurante_id:1, calificacion:-1 }	Compuesto	Reseñas de restaurante ordenadas por calificación
reseñas	{ usuario_id: 1 }	Simple	Reseñas escritas por un usuario
reseñas	{ fecha: -1 }	Simple	sort({ fecha:-1 }) — reseñas recientes
reseñas	{ tags: 1 }	Multikey	find({ tags:'rapido' })
reseñas	{ titulo:'text', comentario:'text' }	Text	$text:{ $search:'excelente servicio' }

Análisis de Rendimiento con explain()

Para validar que cada índice definido es efectivamente utilizado por MongoDB, se ejecutará explain("executionStats") sobre las queries representativas de cada colección. A continuación se documenta la query, el índice esperado y los criterios de aceptación para confirmar que el plan de ejecución es óptimo.

Criterios de aceptación comunes para todos los casos:

Campo explain()	Valor esperado	Qué confirma
winningPlan.stage	IXSCAN	El query usa el índice, no un escaneo completo.
totalDocsExamined	totalDocsReturned	No se examinan documentos innecesarios.
totalKeysExamined	totalDocsReturned	El índice es selectivo para este query.
executionTimeMillis	< 5 ms en dev	Latencia aceptable con datos de prueba.
rejectedPlans	[] vacío o sin COLLSCAN	MongoDB no consideró rutas más lentas.

Caso 1 — Login de usuario (índice simple único)

Índice: `{ email: 1 }` | Colección: `usuarios`

Query:

```
db.usuarios.find(
  { email: "cliente@ejemplo.com" }
).explain("executionStats")
```

Output esperado:

```
winningPlan.stage          → "IDHACK" o "IXSCAN"
winningPlan.indexName      → "email_1"
totalDocsExamined          → 1
totalKeysExamined          → 1
executionTimeMillis        → < 2 ms
```

Interpretación: totalDocsExamined = 1 confirma que MongoDB localiza el usuario directamente sin escanear la colección. Crítico porque este query se ejecuta en cada login.

Caso 2 — Dashboard del propietario (índice compuesto ESR)

Índice: `{ restaurante_id:1, estado:1, fecha_creacion:-1 }` | Colección: `ordenes`

Query:

```
db.ordenes.find(
  {
    restaurante_id: ObjectId("..."),
    estado: "pendiente"
  }
).sort({ fecha_creacion: -1 })
 .explain("executionStats")
```

Output esperado:

```
winningPlan.stage          → "IXSCAN"
winningPlan.indexName      → "restaurante_id_1_estado_1_fecha_creacion_-1"
totalDocsExamined          → ≈ totalDocsReturned
totalKeysExamined          → ≈ totalDocsReturned
executionTimeMillis        → < 5 ms
rejectedPlans              → sin COLLSCAN
```

Interpretación: El índice aplica la regla ESR (Equality -> Sort -> Range). MongoDB filtra por restaurante_id y estado (Equality) y ordena por fecha_creacion sin un sort en memoria (Sort). Si el explain reportara un stage SORT separado, el índice no estaría siendo usado para el ordenamiento.

Caso 3 — Búsqueda geoespacial de restaurantes cercanos

Índice: `{ ubicacion: '2dsphere' }` | Colección: `restaurantes`

Query:

```
db.restaurantes.find({
  ubicacion: {
    $near: {
      $geometry: { type: "Point", coordinates: [-90.5069, 14.6407] },
      $maxDistance: 2000
```

```
    }
  },
  activo: true
}).explain("executionStats")
```

Output esperado:

```
winningPlan.stage          → "GEO_NEAR_2DSPHERE"
winningPlan.indexName      → "ubicacion_2dsphere"
totalDocsExamined          → proporcional al área consultada
executionTimeMillis        → < 10 ms
```

Interpretación: El stage GEO_NEAR_2DSPHERE confirma que MongoDB usa el índice geoespacial. Sin este índice, $near lanza un error, no hay fallback a COLLSCAN para queries geoespaciales, lo que hace este índice obligatorio.

Caso 4 — Menú filtrado por categoría y disponibilidad (índice compuesto)

Índice: `{ restaurante_id:1, categoria:1, disponible:1 }` | Colección: `menu_items`

Query:

```
db.menu_items.find({
  restaurante_id: ObjectId("..."),
  categoria: "principal",
  disponible: true
}).explain("executionStats")
```

Output esperado:

```
winningPlan.stage          → "IXSCAN"
winningPlan.indexName      → "restaurante_id_1_categoria_1_disponible_1"
totalDocsExamined          → ≈ totalDocsReturned
totalKeysExamined          → ≈ totalDocsReturned
```

Interpretación: Los tres campos del índice se usan como filtros de igualdad , máxima selectividad. Si totalKeysExamined fuera mucho mayor que totalDocsReturned, indicaría que el índice no es selectivo para esta combinación de valores.

Caso 5 — Búsqueda de texto en restaurantes

Índice: `{ nombre:'text', descripcion:'text' }` | Colección: `restaurantes`

Query:

```
db.restaurantes.find({
  $text: { $search: "pizza guatemalteca" }
}, {
```

```
  score: { $meta: "textScore" }
}).sort({ score: { $meta: "textScore" } })
 .explain("executionStats")
```

Output esperado:

```
winningPlan.stage          → "TEXT"
winningPlan.indexName      → "nombre_text_descripcion_text"
executionTimeMillis        → < 10 ms
```

Interpretación: El stage TEXT confirma uso del índice de texto. MongoDB solo permite un índice de texto por colección, por eso nombre y descripcion se combinan en uno solo. Si el stage fuera COLLSCAN, $text fallaría con error, también es obligatorio como $near.

Caso 6 — Reseñas por restaurante con multikey (array de tags)

Índice: `{ tags: 1 }` | Colección: `reseñas`

Query:

```
db.resenas.find({
  tags: "rapido"
}).explain("executionStats")
```

Output esperado:

```
winningPlan.stage          → "IXSCAN"
winningPlan.isMultiKey     → true
winningPlan.indexName      → "tags_1"
totalDocsExamined          → ≈ totalDocsReturned
```

Interpretación: isMultiKey: true confirma que MongoDB reconoció el campo como array y aplicó el índice multikey correctamente. Cada elemento del array tags genera una entrada separada en el índice, permitiendo filtrar por cualquier tag sin escanear la colección.

Validación con notablescan activado

Durante el desarrollo, el sistema corre con notablescan activado. Esto garantiza que cualquier query que no use un índice falle inmediatamente con el error:

```
MongoServerError: query requires a table scan
{ code: 291, codeName: "NoTableScan" }
```

Si al implementar un endpoint nuevo en Etapa 02 aparece este error, el procedimiento es:

- [ ] **1.** Correr explain("queryPlanner") para identificar qué campo está causando el COLLSCAN.
- [ ] **2.** Definir el índice faltante en el schema de Mongoose.
- [ ] **3.** Verificar con explain("executionStats") que el nuevo índice produce IXSCAN.
- [ ] **4.** Confirmar que totalDocsExamined ≈ totalDocsReturned antes de hacer merge.

Propuesta de shard key y análisis de escalabilidad

1. Principio Rector: Colocation

El criterio central para elegir las shard keys es la colocation: mantener en el mismo shard los datos que se consultan juntos. En este sistema el eje de acceso más frecuente es el restaurante, el dashboard del propietario consulta sus órdenes, su menú y sus reseñas de forma conjunta. Si esos documentos están en shards distintos, cada query se convierte en una *scatter-gather operation* con latencia de red adicional en cada petición.

2. Shard Keys Propuestas

Colección	Shard Key	Tipo	Justificación
ordenes	{ restaurante_id: 1, fecha_creacion: 1 }	Compuesto por rango	Colocaliza órdenes con el menú del mismo restaurante. fecha_creacion evita el hotspot de escritura en restaurantes muy activos.
menu_items	{ restaurante_id: 1 }	Por rango	Colocaliza el menú con las órdenes del mismo restaurante. Queries de checkout son single-shard.
reseñas	{ restaurante_id: 1, fecha: -1 }	Compuesto por rango	El pipeline de Top Restaurantes filtra por restaurante_id; si reseñas y restaurante están en el mismo shard, el $lookup es local.
usuarios	{ _id: 'hashed' }	Hashed	Sin patrón de acceso localizado. Hashed distribuye uniformemente y evita hotspots.
restaurantes	{ _id: 'hashed' }	Hashed	Sin patrón concentrado en fase inicial. Permite migrar a zone sharding por región si el sistema escala a múltiples países.

3. Shard Keys Descartadas

Por qué se rechazaron las alternativas:

Shard Key Descartada	Colección	Problema
{ _id: 'hashed' }	ordenes	Rompe colocation con restaurantes. El dashboard del propietario se convierte en scatter-gather query a todos los shards.
{ estado: 1 }	ordenes	Cardinalidad de solo 5 valores. MongoDB crearía 5 chunks que nunca balancearían.
{ fecha_creacion: 1 } solo	ordenes	Hotspot permanente: todos los inserts van al shard más reciente, los demás quedan inactivos.
{ usuario_id: 1 }	ordenes	Distribución desigual: un restaurante popular genera muchas más órdenes que un cliente casual.

4. Reutilización de Índices como Shard Key Prefix

MongoDB requiere que exista un índice cuyo prefijo coincida con la shard key. Los índices compuestos ya definidos en el esquema cumplen esto sin necesidad de índices adicionales:

Índice existente	Colección
{ restaurante_id:1, estado:1, fecha_creacion:-1 }	ordenes
{ restaurante_id:1, categoria:1, disponible:1 }	menu_items
{ restaurante_id:1, calificacion:-1 }	reseñas

Habilitar sharding en Atlas no requiere ningún cambio en el esquema ni índices extra. El diseño fue pensado para escalar sin rediseño.

5. Nota sobre Sharding en Fase Inicial

En un deployment de escala inicial, habilitar sharding no es necesario y añade complejidad operacional (mongos router, config servers, chunk balancer). La propuesta se documenta ahora porque:

- Una vez sharded con una shard key incorrecta, cambiarla requiere migración completa de datos es un error muy costoso de corregir a posteriori.
- Los índices de Etapa 01 ya son compatibles con las shard keys propuestas, por lo que el esquema está listo para escalar sin rediseño cuando el volumen lo justifique.

- Atlas facilita habilitar sharding cuando el tier lo permite (M30+), pero la arquitectura de datos debe planificarse desde el inicio.

Arquitectura del Sistema y stack tecnologico

Capa	Tecnología	Rol en el sistema
Monorepo	Turborepo	Gestión del monorepo. Caché de builds, pipelines paralelas para backend y frontend.
Frontend	React + Vite	SPA. Consume la REST API via Axios/Fetch. Sin lógica de negocio — solo presentación y estado local.
Backend	NestJS	Framework Node.js. Provee la arquitectura MVC, inyección de dependencias, decoradores y módulos. Expone la REST API.
Base de datos	MongoDB Atlas	Replica set 3 nodos. Transacciones ACID, índices, aggregation framework, GridFS.
Archivos	GridFS	Almacenamiento de imágenes (portadas y platillos) y exportaciones de reportes en PDF/CSV.
ODM	Mongoose	Validación de schemas a nivel de aplicación. Modelos tipados que mapean las colecciones.
Autenticación	JWT + Passport	Guards de NestJS protegen endpoints. Roles: cliente, propietario, admin.